

Capítulo 4/6 · Faltan 4 lecciones

Trabajar con valores duplicados y ausentes

1 h 10 min



Rellenar los valores categóricos ausentes

Teoría

Antes de sumergirnos en cómo rellenar los valores ausentes, necesitamos un momento para analizar los dos tipos de variables con las que trabajarás como profesional de datos: cuantitativas y categóricas.

Rellenar los valores categóricos ausentes



► Transcripción del video

Pregunta

En la última lección determinamos que en nuestras columnas de dirección de correo electrónico y fuente de tráfico hay valores `NaN`. ¿Qué tipo de variables son estas columnas?

☐ Tanto `'source'` como `'email'` son variables cuantitativas.

☐ `'source'` es categórica y `'email'` es cuantitativa.

☐ Ambas columnas son categóricas.

☐ `'source'` es cuantitativa y `'email'` es categórica.

☐ Tanto `'source'` como `'email'` son categóricas.



Incorrecto, pero no te desanimes: el aprendizaje lleva tiempo

Tanto `'source'` como `'email'` son columnas categóricas. De hecho, cada columna en el conjunto de datos de fuente de tráfico es categórica. En esta lección aprenderemos algunas formas efectivas de rellenar los valores categóricos ausentes.

Usar `fillna()` no es la única forma en que podemos rellenar los valores ausentes con cadenas vacías. Por cierto, también podemos hacerlo directamente al leer los datos mediante `read_csv()`.

El parámetro `keep_default_na=`

Si observas los datos de texto sin procesar en el archivo `visit_log.csv`, encontrarás que los valores ausentes están representados por la ausencia de texto.

En otras palabras, la ausencia de texto en `visit_log.csv` se interpreta como `NaN`. Consulta el archivo CSV en la pestaña de la lección.

Pero podemos hacer que `read_csv()` lea la ausencia de texto como cadenas vacías en lugar de `NaN`, configurando el parámetro `keep_default_na=` en `False`. Probémoslo en nuestro conjunto de datos:

```
import pandas as pd

df_logs = pd.read_csv('/datasets/visit_log.csv', keep_default_na=False)

print(df_logs.head())
print()
df_logs.info()
```

	user_id	source	email	purchase
0	7141786820	other		0
1	5644686960	email	c129aa540a	0
2	1914055396	context		0
3	4099355752	other		0
4	6032477554	context		1

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200000 entries, 0 to 199999
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   user_id     200000 non-null  int64
1   source      200000 non-null  object
2   email       200000 non-null  object
3   purchase    200000 non-null  int64
dtypes: int64(2), object(2)
memory usage: 6.1+ MB
```

En la impresión podemos ver que los valores ausentes son strings vacíos. Ahora `info()` ya no detectará ningún valor nulo, esto significa que depende de nosotros reconocer que `' '` representa un valor ausente a medida que avanzamos con nuestro análisis.

Ten en cuenta que establecer `keep_default_na=False` convierte todos los valores ausentes en strings vacíos, incluso para las columnas numéricas. Esto hace que las columnas numéricas se lean como strings cuando tienen valores ausentes. Así que asegúrate de usar solo `keep_default_na=False` cuando desees que todos los valores ausentes en cada columna se lean como strings vacíos.

En el caso de `visit_log.csv`, nos conviene hacer esto porque todas nuestras columnas son categóricas. ¡Pudimos leer los datos y reemplazar todos los valores ausentes con tan solo una pequeña línea de código!

Práctica guiada

Los valores `NaN` en la columna `'email'` sustituyen a las direcciones de correo electrónico de los usuarios que no se suscribieron al boletín de la tienda. Ya que no hay forma de averiguar sus direcciones de correo electrónico, no podemos rellenar manualmente los valores ausentes con datos significativos.

Pero podemos rellenarlos con un **valor por defecto** para representar los correos electrónicos ausentes. Sustituyamos los valores ausentes en la columna `'email'` por el string vacío `' '` como valor por defecto.

1. Utiliza el método `fillna()` para sustituir los valores ausentes en `'email'` por strings vacíos.
2. Imprime las cinco primeras filas del `DataFrame`.

Código

PYTHON

```
1 import pandas as pd
```



¿No es un número? ¡Ya no! Ahora, cuando imprimimos el `DataFrame`, no vemos nada para los valores de `'email'` ausentes porque con el string vacío no hay

nada que imprimir.

 Pista



Mostrar la solución

Resultado

	user_id	source	email	purchase
0	7141786820	other		0
1	5644686960	email	c129aa540a	0
2	1914055396	context		0
3	4099355752	other		0
4	6032477554	context		1

Debido a que no podemos conocer las direcciones de correo electrónico de los visitantes que nunca proporcionaron una, tiene sentido usar el string vacío para esos valores ausentes. Pero, ¿qué pasa con los valores de fuente de tráfico ausentes?

Esas visitas vienen de correos electrónicos, así que vamos a reemplazar manualmente los valores vacíos en 'source' por 'email'.

Para comenzar:

- Lee los datos usando `keep_default_na=False`.
- Imprime todos los valores únicos en la columna 'source'.

Esto te permitirá confirmar cuántas entradas aparecen como vacías y deben ser corregidas.

Código

PYTHON

```
1 import pandas as pd
2
```



Tenemos 5 valores distintos para la categoría de fuente de tráfico, y los valores ausentes ahora se representan con ''. El valor para las fuentes de correo electrónico de marketing es el string 'email'. De acuerdo con nuestro análisis, queremos convertir todos los valores de '' a 'email'.

 Pista



Mostrar la solución

Resultado

```
['other' 'email' 'context' '' 'undef']
```

¡Genial! ¡Ahora vamos a arreglar nuestros valores ausentes!

1. Utiliza `replace()` para reemplazar los valores ausentes en la columna `'source'` por el string `'email'`.
2. Verifica tu trabajo llamando al método `unique()` en la columna `'source'` e imprime los resultados.

Código

PYTHON

```
1 import pandas as pd
```



¡Lo conseguiste! No hay más valores `''` en la columna `'source'`. Todas nuestras fuentes de visitas están contabilizadas. Ahora que hemos rellenado los valores ausentes, podemos calcular la tasa de conversión para cada fuente. Para ello, tendremos que utilizar la función `groupby()` para la agregación. Vamos a repasar rápidamente cómo utilizarlo.

Pista



Mostrar la solución

Resultado

```
['other' 'email' 'context' 'undef']
```

Actividad práctica

Ejercicio 1

Para calcular la tasa de conversión de cada fuente de tráfico, primero determina cuántas visitas hubo de cada fuente.

Para encontrar el número total de visitas de cada fuente de tráfico, utiliza `groupby()` para agrupar los datos por la columna `'source'`, luego cuenta el

número de valores en la columna 'user_id' del DataFrame agrupado. Asigna el resultado en la variable `visits` y luego imprímelo.

El precódigo ya contiene el trabajo que realizaste para rellenar los valores ausentes.

Código

PYTHON

```
1 import pandas as pd
2
3 df_logs = pd.read_csv('/datasets/visit_log.csv',
4                       keep_default_na=False)
```



Sprint 3: Manipulación de datos (Data Wrangling)



✓ La mayor fuente de tráfico es aparentemente 'other' (otros). MISTERIOSO...



Pista



Mostrar la solución

Resultado

```
source
context      52032
email        13953
other        133834
undef         181
Name: user_id, dtype: int64
```

Ejercicio 2

A continuación, determina el número de visitas en las que se realizó una compra para cada fuente, calculando la suma de la columna 'purchase' para cada grupo de fuente. Posteriormente, asigna los resultados a la variable `purchases` e imprímelos.

Código

PYTHON

```
1 import pandas as pd
2
3 df_logs = pd.read_csv('/datasets/visit_log.csv',
4                       keep_default_na=False)
```



```
4 df_logs['source'] = df_logs['source'].replace('', 'email')
```

✓ "Otros" también se las arregló para conseguir el mayor número de compras.

🔍 Pista



Mostrar la solución

Resultado

```
source
context    3029
email      1021
other       8041
undef         12
Name: purchase, dtype: int64
```

Ejercicio 3

Calcula la tasa de conversión para cada fuente de tráfico, guarda los resultados en `conversion`, e imprímelos. La tasa de conversión es la proporción de visitas en las que se realizó una compra, o sea `purchases / visits`. El precódigo contiene las visitas y compras de tu trabajo previo.

Código

PYTHON

```
1 import pandas as pd
2
3 df_logs = pd.read_csv('/datasets/visit_log.csv',
4                       keep_default_na=False)
5
6 df_logs['source'] = df_logs['source'].replace('', 'email')
7
8 visits = df_logs.groupby('source')['user_id'].count()
```

✓ Aunque "otros" tiene más compras en general, parece que el correo electrónico tiene la mejor tasa de conversión. En unas pocas líneas de código, has conseguido calcular la métrica más importante del marketing.

🔍 Pista



Mostrar la solución

Resultado

```
source
context    0.058214
```

email	0.073174
other	0.060082
undef	0.066298



Valoración de la lección